

ProLoop: Progress-Aware Closed-Loop Framework via Structured State Relations for Long-Horizon Robotic Task Planning

Anonymous Author(s)

Abstract—Long-horizon robotic tasks are commonly decomposed into semantic subgoals to simplify planning. However, subgoal progress in LLM-based planning is typically inferred implicitly from interaction histories. Inaccurate estimates can misguide execution and allow local errors to propagate. To address this, we propose ProLoop, a Progress-Aware Closed-Loop framework that derives explicit subgoal progress from structured relations between desired and execution states. A learned State-Relation Monitor assesses these relations, which are then aggregated into subgoal progress as ongoing, deviated, or completed. The resulting progress guides execution adaptation, enabling reliable progress-aware control throughout execution. Experiments on AgentBoard ALFWorld and a real-world robotic platform show that ProLoop improves success rate by 12.94% overall and by 15.46% on Hard tasks, while reducing environment interactions by 32.3%. Project page (anonymized): <https://proloopplanning.github.io/>

I. INTRODUCTION

Recent advances in robot task planning increasingly leverage large language models (LLMs) to generate coherent multi-step plans [1], [2]. Despite strong performance on complex tasks, LLM-based planning remains susceptible to hallucinations and logical inconsistencies [3], [4]. To manage long-horizon planning complexity, hierarchical methods decompose tasks into semantic subgoals, allowing each stage to focus on a simpler intermediate objective [5]–[7]. However, during subgoal execution, subgoal progress is still commonly inferred implicitly by the LLM from accumulated interaction trajectories and multi-source information [8]. Inaccurate progress estimates can therefore misguide execution, allowing local errors to accumulate and propagate across subsequent subgoals.

Building on these observations, recent studies [8], [9] address this problem by introducing explicit checks at selected checkpoints to assess the current execution state and then support replanning (see Fig. 1). These checks form a feedback loop that reduces reliance on implicit progress inference by the LLM. However, such checkpoint-based feedback cannot timely respond to changes in subgoal progress, since reassessment occurs only at the next checkpoint. During robotic execution, this delay can allow erroneous actions to alter the environment state, making recovery costly or even infeasible [10], [11]. We therefore ask: **How can robots maintain reliable progress awareness throughout subgoal execution to enable timely closed-loop adaptation?**

A useful intuition comes from how people perform everyday tasks. People assess progress by relating the current state to the intended outcome before deciding what to do next. Following this intuition, we define robotic subgoal progress by aggregating structured relations between desired subgoal

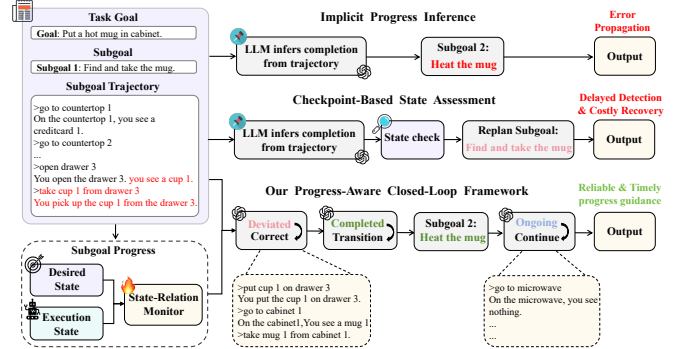


Fig. 1. Compared with existing approaches, ProLoop explicitly derives subgoal progress from the relation between desired and execution states, distinguishes *ongoing*, *deviated*, and *completed* execution, and uses the resulting progress to support timely closed-loop adaptation.

conditions and the evolving execution state into *ongoing*, *deviated*, or *completed*.

This formulation raises two key challenges. **Constructing comparable desired and execution states.** A semantic subgoal specifies its desired outcome in natural language, whereas the execution state must be reconstructed from evolving actions, observations, perception, and contextual information. Making the two states comparable is difficult, but essential for accurate subgoal progress assessment. **Using progress to guide execution without sacrificing LLM flexibility.** Even with reliable progress assessment, different progress states call for different responses. Excessive intervention during ongoing execution may disrupt useful LLM reasoning and action generation, whereas insufficient intervention after deviation may allow errors to persist. Moreover, completion should be leveraged beyond stage termination, as its associated outcomes must be selectively transferred to support subsequent planning.

To address these challenges, we propose ProLoop, a Progress-Aware Closed-Loop framework for long-horizon robotic task planning. For each active subgoal generated by the **LLM-Based Subgoal Planner**, **Structured State Construction** first converts the subgoal’s desired outcome and evolving execution information into comparable desired and execution states under a shared task-state schema. A learned **State-Relation Monitor** then assesses these condition-level relations and derives the active subgoal progress as ongoing, deviated, or completed. The resulting progress drives **Progress-Guided Adaptation**: ongoing execution preserves the underlying LLM’s reasoning process, deviation triggers targeted correction, and completion permits transition to the next subgoal. Additionally, upon completion, **Completion-**

Gated Memory Update uses planner-generated retention cues to selectively summarize task-relevant execution information for subsequent planning.

To assess the effectiveness of ProLoop, we conduct experiments on the AgentBoard [12] ALFWorld household planning benchmark, which contains six task categories with easy and hard difficulty levels. Empirical results demonstrate improved overall performance, with especially substantial gains on hard tasks. We further validate ProLoop on a physical robot. Our contributions are threefold:

- 1) We propose **ProLoop**, a unified progress-aware closed-loop framework for robotic task planning that integrates structured state construction, state-relation monitoring, and progress-guided adaptation.
- 2) We define explicit subgoal progress through structured relations between desired and execution states, distinguishing *ongoing*, *deviated*, and *completed* progress.
- 3) We demonstrate that ProLoop improves task success rate and progress rate on ALFWorld while requiring fewer environment interactions. We further validate its feasibility on a physical robot.

II. RELATED WORK

A. LLMs for Task Planning

Recent studies have leveraged the reasoning and long-horizon planning capabilities of LLMs for robotic tasks [3]. LID [13] formulates observations, goals, and interaction histories as sequential representations for policy prediction, while ReAct [14] interleaves reasoning and action generation. Code-based methods, including Code as Policies [15] and ProgPrompt [1], further represent plans as executable programs to improve structural consistency and interpretability. In embodied settings, SayCan [16] grounds language-based reasoning in robot affordances, and LLM-Planner [17] extends LLM-based planning to embodied decision-making tasks. These methods generate and ground multi-step plans, but long-horizon dependencies and growing interaction histories motivate more structured task organization across successive subgoals.

B. Hierarchical Task Planning

Hierarchical approaches reduce long-horizon complexity by organizing tasks around intermediate semantic objectives. HiP [5] coordinates language planning, visual prediction, and low-level control across multiple abstraction levels, whereas DELTA [6] uses LLM-generated subgoals to construct smaller problems for an automated planner. STEP Planner [18] organizes planning through a hierarchical subgoal tree, while HiAgent [7] treats semantic subgoals as working-memory chunks and compresses completed trajectories. Together, these methods establish subgoals as effective units for task decomposition and context organization. However, subgoal progress is still often inferred implicitly by the LLM, making execution prone to error propagation.

C. Feedback-Aware Task Planning

Feedback-aware methods incorporate execution information to improve planning robustness. Inner Monologue [19] integrates environment feedback into closed-loop language planning, while AdaPlanner [9] uses predicted feedback and assertions to refine plans during execution. CoEx [8] verifies subgoal experiences to update an adaptive belief state, and LLaMAR [20] combines subtask verification with execution correction. KnowLoop [21] further detects whether executed subgoals have succeeded using an uncertainty-aware failure detector. Collectively, these methods support plan refinement, belief updating, verification, and failure detection.

In summary, prior work has advanced LLM planning through action generation, hierarchical decomposition, and feedback-based adaptation. However, subgoal progress is often inferred implicitly or assessed through discrete feedback mechanisms, leaving continuous progress awareness underexplored.

III. PRELIMINARIES

A. Planning Framework

A task is characterized by a tuple $\mathcal{P} = \langle G, S, O, A, T \rangle$, where G denotes the goal condition, S and O represent the state and observation spaces, A is the set of available actions, and $T : S \times A \rightarrow S$ specifies the state transition induced by action execution. Planning aims to obtain a policy that drives the system from an initial state to a state satisfying G . In LLM-based planning, these task elements are commonly expressed through a natural-language description D provided to the model.

B. Hierarchical Subgoal Execution

In hierarchical LLM planning, a long-horizon task is executed through a sequence of semantic subgoals. At stage k , the planner generates an active subgoal g_k as an intermediate objective toward the overall goal G . Executing g_k involves multiple environment interactions, forming a local trajectory $\tau_k^t = [(a_{k,1}, o_{k,1}), \dots, (a_{k,t}, o_{k,t})]$. Let m_i denote the information retained from the trajectory of a previous subgoal g_i . The cross-subgoal context before executing g_k is therefore $M_k = [(g_1, m_1), \dots, (g_{k-1}, m_{k-1})]$. Accordingly, the working context of the active subgoal is represented as $W_k^t = [G, M_k, g_k, \tau_k^t]$.

IV. METHODOLOGY

In this section, we detail our proposed framework, **ProLoop**. As illustrated in Fig. 2, the framework forms a closed loop around each active subgoal through structured state construction, state-relation monitoring, progress-guided adaptation, and completion-gated memory update. The LLM-Based Subgoal Planner generates an active semantic subgoal and retention cues. Structured State Construction represents the desired subgoal outcome and evolving execution information under a shared task-state schema. The State-Relation Monitor compares each desired condition against the current execution state and classifies the resulting relation as *supported*, *unresolved*, or *contradicted*. These relations

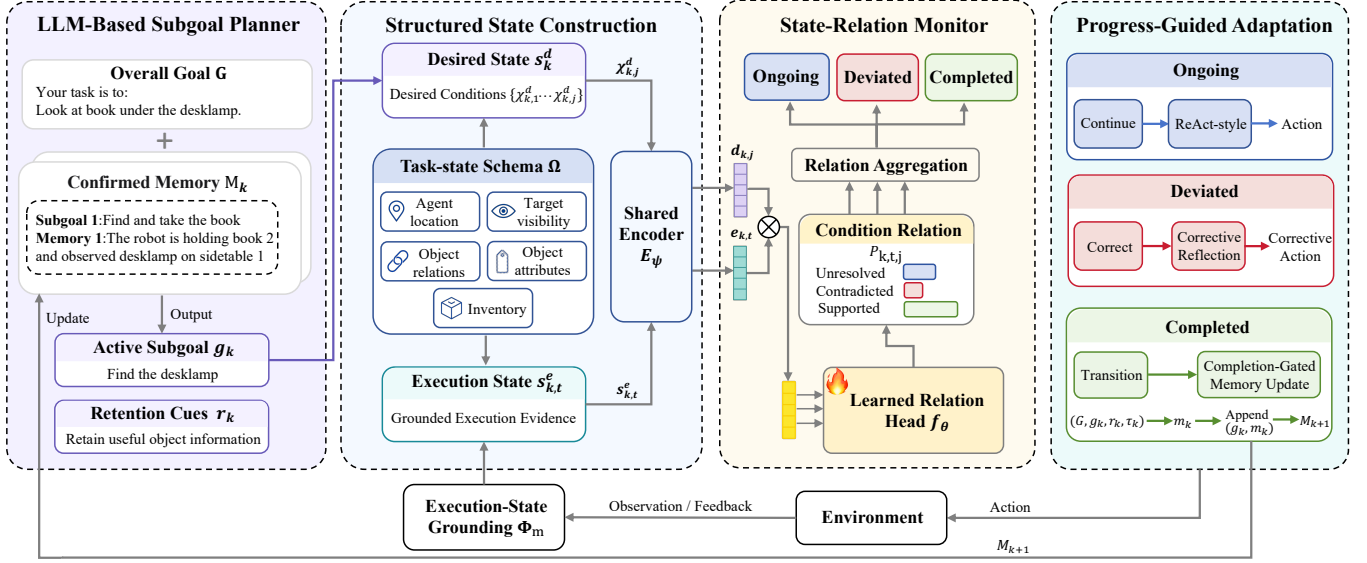


Fig. 2. Overview of ProLoop. The LLM-Based Subgoal Planner generates the active subgoal and retention cues. Structured State Construction derives comparable desired and execution states, the State-Relation Monitor infers subgoal progress from their structured relations, and Progress-Guided Adaptation regulates continuation, targeted correction, or subgoal transition, with memory updated upon completion.

are then aggregated into *ongoing*, *deviated*, or *completed* progress. Progress-Guided Adaptation maps the resulting progress state to continuation, correction, or subgoal transition. Upon completion, Completion-Gated Memory Update uses the retention cues to summarize the local trajectory into a new memory entry before the next subgoal is generated.

A. LLM-Based Subgoal Planner

At stage k , the LLM-based planner generates an active semantic subgoal g_k and retention cues r_k :

$$(g_k, r_k) = P_{\text{LLM}}(G, M_k, I_P), \quad (1)$$

where G and M_k are defined in Sec. III, and I_P contains the planning instruction, in-context demonstrations, and output format. The subgoal g_k specifies the current intermediate objective, whereas r_k indicates information that may be useful to subsequent subgoals. ProLoop generates one active subgoal at a time, allowing later planning decisions to condition on confirmed outcomes from earlier stages.

B. Structured State Construction

Structured State Construction converts the desired subgoal outcome and evolving execution information into comparable desired and execution states under a shared task-state schema Ω . It produces a desired state s_k^d and an execution state s_k^e for subsequent relation assessment by the monitor.

Desired-State Construction. The schema Ω contains general task-state field types, including agent location, target visibility, object attributes, object relations and inventory. Given the active subgoal, an LLM-based constructor generates the following desired state:

$$s_k^d = C_{\text{LLM}}(g_k, \Omega, I_C), \quad (2)$$

where I_C specifies the construction instruction. The resulting state is a sparse set of J_k desired conditions, $s_k^d = \{\chi_{k,j}^d\}_{j=1}^{J_k}$.

Each condition is represented by the following tuple:

$$\chi_{k,j}^d = \langle f_{k,j}, b_{k,j}, v_{k,j}^d \rangle. \quad (3)$$

The three tuple elements denote, respectively, the field type, entity binding, and desired value. Desired conditions may retain category-level bindings, whereas execution states record grounded instances (e.g., *mug* versus *mug* 1).

Execution-State Grounding. Let $o_{k,t}$ denote the observation and available execution feedback after action $a_{k,t}$. A modality-specific grounding function Φ_m extracts task-relevant fields:

$$\Delta s_{k,t}^e = \Phi_m(a_{k,t}, o_{k,t}, g_k, \Omega). \quad (4)$$

The active subgoal guides task-relevant grounding, while Ω specifies the output schema. At task start, $s_{1,0}^e = \emptyset$; for $k > 1$, $s_{k,0}^e = s_{k-1,T_{k-1}}^e$, where T_{k-1} is the final step of g_{k-1} . The execution state is updated as follows:

$$s_{k,t}^e = \mathcal{U}(s_{k,t-1}^e, \Delta s_{k,t}^e), \quad (5)$$

where $\mathcal{U}$ incorporates new information and removes invalidated facts. For constrained text environments, Φ_m uses deterministic parsing over agent-visible observations and action feedback. For physical execution, visual observations are grounded by a VLM with robot and skill feedback. Persistent facts are retained unless invalidated, while transient fields are refreshed from current observations and unavailable values are marked unknown. Let $\text{Inv}(o_{k,t})$ indicate an invalid action. Such feedback directly yields *deviated* progress.

C. State-Relation Monitor

As illustrated in Fig. 3, the monitor compares each desired condition with the current execution state and aggregates the resulting relations into subgoal progress. A *supported* relation indicates that available evidence establishes the desired condition. An *unresolved* relation indicates that the condition

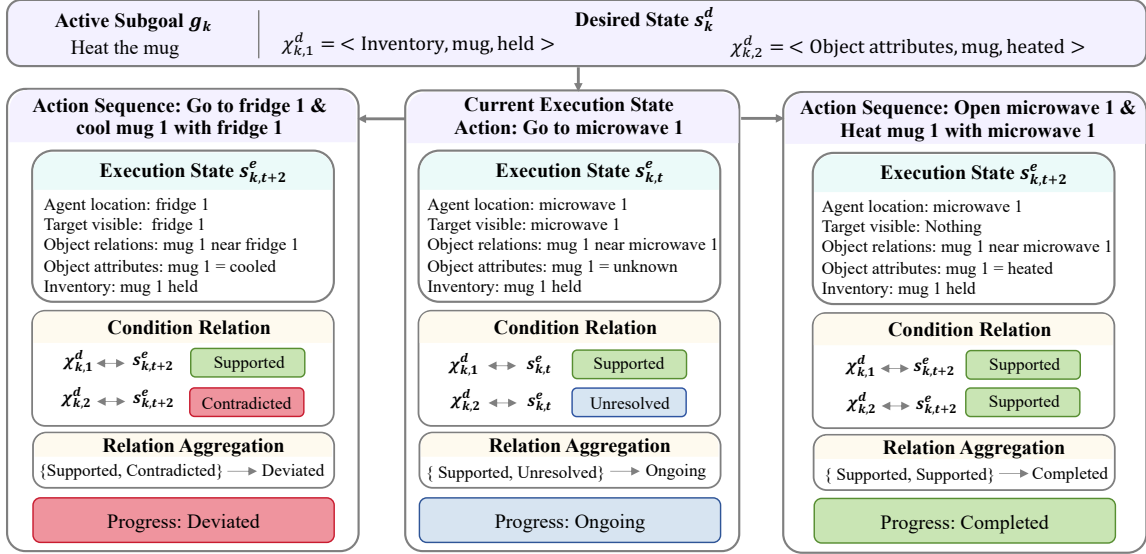


Fig. 3. Worked example of state-relation monitoring and progress derivation. For the active subgoal *Heat the mug*, desired conditions are compared with the corresponding execution state, and the resulting relations are aggregated to derive *ongoing*, *deviated*, or *completed* subgoal progress.

is not yet established and no conflicting evidence is available. A *contradicted* relation indicates explicit evidence of a task-relevant conflict with the desired condition. For instance, when the desired attribute is heated, an unknown thermal-state value is unresolved, whereas an explicitly recorded cooled value is contradicted.

The execution state $s_{k,t}^e$ is serialized in a fixed schema order and encoded once per interaction, while each desired condition $\chi_{k,j}^d$ is encoded individually using the same pre-trained semantic encoder:

$$\mathbf{d}_{k,j} = E_\psi(\text{Ser}(\chi_{k,j}^d)), \quad \mathbf{e}_{k,t} = E_\psi(\text{Ser}(s_{k,t}^e)), \quad (6)$$

where $\text{Ser}(\cdot)$ converts a structured input into a canonical textual sequence: execution-state fields follow the fixed schema order, while each desired condition preserves its field type, entity binding, and target value. For each desired condition, we construct a pairwise relation representation using the two embeddings, their absolute difference, and their element-wise interaction to capture condition–state compatibility:

$$\mathbf{x}_{k,t,j} = [\mathbf{d}_{k,j}; \mathbf{e}_{k,t}; |\mathbf{d}_{k,j} - \mathbf{e}_{k,t}|; \mathbf{d}_{k,j} \odot \mathbf{e}_{k,t}]. \quad (7)$$

Here, $[\cdot; \cdot]$ denotes concatenation and $\odot$ denotes the element-wise product. A shared lightweight MLP relation head f_θ is trained to assess condition–state compatibility using these pairwise features, with category-to-instance correspondence learned implicitly through supervision from the three relation labels. The head predicts the relation probabilities:

$$\mathbf{p}_{k,t,j} = \text{softmax}(f_\theta(\mathbf{x}_{k,t,j})). \quad (8)$$

The probability vector contains the following components:

$$\mathbf{p}_{k,t,j} = [p_{k,t,j}^{\text{sup}}, p_{k,t,j}^{\text{unr}}, p_{k,t,j}^{\text{con}}]. \quad (9)$$

The entries correspond to the predicted probabilities of *supported*, *unresolved*, and *contradicted*, respectively. The predicted condition relation is determined as follows:

$$\ell_{k,t,j} = \arg \max_{c \in \mathcal{Q}} p_{k,t,j}^c, \quad \mathcal{Q} = \{\text{sup}, \text{unr}, \text{con}\}. \quad (10)$$

We derive subgoal progress by aggregating the predicted condition relations and checking for invalid-action feedback:

$$\rho_{k,t} = \begin{cases} \text{deviated}, & \text{Inv}(o_{k,t}) \vee \exists j : \ell_{k,t,j} = \text{con}, \\ \text{completed}, & \neg \text{Inv}(o_{k,t}) \wedge \forall j : \ell_{k,t,j} = \text{sup}, \\ \text{ongoing}, & \text{otherwise.} \end{cases} \quad (11)$$

We adopt a conservative aggregation rule: contradictions trigger correction, while transition requires all desired conditions to be supported.

D. Progress-Guided Adaptation

The derived progress state regulates the underlying executor by selecting one of three control actions:

$$\mathcal{A}(\rho_{k,t}) = \begin{cases} \text{CONTINUE}, & \rho_{k,t} = \text{ongoing}, \\ \text{CORRECT}, & \rho_{k,t} = \text{deviated}, \\ \text{TRANSITION}, & \rho_{k,t} = \text{completed}. \end{cases} \quad (12)$$

Continue. For an ongoing subgoal, ProLoop preserves the underlying ReAct-style think–act loop without introducing an additional intervention prompt. The executor continues reasoning from the active subgoal, cross-subgoal memory, and local trajectory, with each interaction re-evaluated by the State-Relation Monitor.

Correct. For a deviated subgoal, ProLoop constructs a correction prompt from the detected conflicting execution information and available failure feedback. The prompt is injected into the executor’s working context to elicit a *Corrective Reflection*, which is retained while the subgoal remains deviated and guides the generation of a *Corrective Action* without changing the active subgoal. Each resulting interaction is re-evaluated by the State-Relation Monitor, and the corrective reflection is removed once the progress leaves the *deviated* state.

Transition. For a completed subgoal, ProLoop terminates the current execution stage and invokes the Completion-

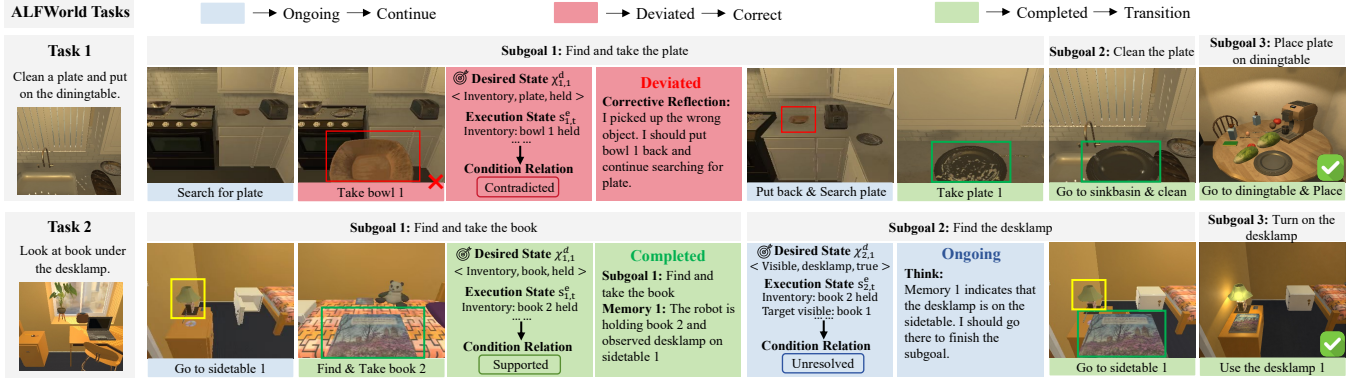


Fig. 4. Representative ALFWorld executions with ProLoop. Ongoing progress allows execution to continue without additional intervention, deviated progress triggers targeted correction, and completed progress enables subgoal transition and memory update.

Gated Memory Update in Sec. IV-E to summarize the completed local trajectory into a memory entry. If the overall goal remains unsatisfied, control returns to the planner to generate g_{k+1} ; otherwise, execution terminates.

E. Completion-Gated Memory Update

Cross-subgoal memory is updated only after the derived progress state indicates completion. An LLM summarizes the complete local trajectory τ_k of g_k , guided by the retention cues r_k to retain task-relevant information:

$$m_k = S_{\text{LLM}}(G, g_k, r_k, \tau_k, I_M), \quad (13)$$

where I_M specifies the memory-update instruction. The retention cues guide the summary but do not prevent newly discovered task-relevant information from being retained.

The new subgoal-memory pair is appended to the existing context to support subsequent subgoal planning:

$$M_{k+1} = M_k \oplus (g_k, m_k). \quad (14)$$

Here, $\oplus$ appends a new entry without merging previous memories. Only the cue-guided summary is carried forward; the full local trajectory is not appended, reducing cross-subgoal context growth.

F. Offline Relation Learning

The State-Relation Monitor is trained offline using trajectories collected only from training environments; no OOD evaluation instances are used for relation-head training. Human-written annotation guidelines define the semantics of the three condition relations. Guided by these guidelines, Qwen3.8-Max serves as the offline annotator and assigns each condition-state pair $(\chi_{k,j}^d, s_{k,t}^e)$ a relation label in $\mathcal{Q} = \{\text{sup}, \text{unr}, \text{con}\}$. A randomly sampled subset of the generated annotations is manually verified for consistency with the same guidelines. The resulting labels provide condition-level supervision for the relation head; the LLM annotator is used only offline and is not involved during execution.

Given the predicted relation distribution $\mathbf{p}_{k,t,j}$, the relation head is optimized using weighted cross-entropy:

$$\mathcal{L}_{\text{rel}} = - \sum_{k,t,j} \sum_{c \in \mathcal{Q}} w_c y_{k,t,j}^c \log p_{k,t,j}^c, \quad (15)$$

where $y_{k,t,j}^c$ is the annotated target and w_c compensates for class imbalance. We freeze the pretrained semantic encoder and train only the MLP relation head.

V. EXPERIMENTS

In this section, we evaluate ProLoop on the AgentBoard ALFWorld benchmark under different task difficulties and LLM backbones. The results demonstrate consistent improvements in overall task performance, with particularly pronounced gains on Hard tasks involving more subgoals.

A. Experimental Setup

Environment and Dataset. We conduct our evaluation on the out-of-distribution (OOD) split of ALFWorld provided by AgentBoard [12], [22]. The benchmark contains six household task categories: *Pick & Place*, *Examine in Light*, *Clean & Place*, *Heat & Place*, *Cool & Place*, and *Pick Two & Place*, comprising 134 unseen instances in total. Following AgentBoard’s difficulty labels, the evaluation set contains 24 Easy and 110 Hard tasks, with three and four benchmark-defined subgoals, respectively. Each episode is capped at 30 environment interactions, and all methods follow the same data split, interaction budget, and evaluation protocol for a consistent comparison across all methods.

Compared Methods. We compare ProLoop with four representative paradigms for LLM-based long-horizon planning: **1) Plan-Only:** AgentBoard’s VanillaAgent directly generates environment actions without explicit reasoning. **2) Reason-Only:** ReAct [14], which interleaves step-wise reasoning and action generation. **3) Reason-Hierarchical:** HiAgent [7], which organizes long interaction histories through semantic subgoals and hierarchical working memory. **4) Reason-Replan:** AdaPlanner [9], which adapts and refines plans using closed-loop execution feedback. We evaluate the methods using Qwen3-14B and GPT-5.6 as LLM backbones. All ALFWorld experiments are conducted over three runs with different random seeds for each method.

Metrics. We evaluate all methods from three complementary perspectives:

- **Overall Task Performance.** Success Rate (SR) measures final task completion, Progress Rate (PR) measures the maximum fraction of benchmark-defined

TABLE I

PERFORMANCE OF PROLOOP AND BASELINES ON THE AGENTBOARD ALFWORLD OOD SPLIT. RESULTS FOR BOTH BACKBONES ARE REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER THREE RUNS WITH DIFFERENT RANDOM SEEDS.

Method	Overall (%)			Easy (%)		Hard (%)	
	SR $\uparrow$	PR $\uparrow$	GA $\uparrow$	SR $\uparrow$	PR $\uparrow$	SR $\uparrow$	PR $\uparrow$
<i>Model: Qwen3-14B</i>							
Plan-Only	57.71 $\pm$ 2.46	67.77 $\pm$ 1.42	82.02 $\pm$ 1.29	81.94 $\pm$ 1.96	84.26 $\pm$ 1.73	52.42 $\pm$ 3.09	64.17 $\pm$ 1.98
Reason-Only	83.58 $\pm$ 1.06	91.21 $\pm$ 0.44	93.18 $\pm$ 0.38	94.44 $\pm$ 5.20	95.37 $\pm$ 4.72	81.21 $\pm$ 1.87	90.30 $\pm$ 0.95
Reason-Hierarchical	69.90 $\pm$ 1.41	79.71 $\pm$ 1.41	94.53 $\pm$ 0.06	90.28 $\pm$ 1.96	91.20 $\pm$ 0.65	65.45 $\pm$ 1.96	77.20 $\pm$ 1.75
Reason-Replan	75.62 $\pm$ 2.54	83.46 $\pm$ 1.56	91.88 $\pm$ 1.08	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	70.30 $\pm$ 3.09	79.85 $\pm$ 1.90
Ours	96.52$\pm$0.70	97.39$\pm$0.18	98.23$\pm$0.05	95.83 $\pm$ 0.00	98.61 $\pm$ 0.00	96.67$\pm$0.86	97.12$\pm$0.21
<i>Model: GPT-5.6</i>							
Plan-Only	84.33 $\pm$ 3.39	90.90 $\pm$ 2.08	94.54 $\pm$ 0.25	93.06 $\pm$ 5.20	93.98 $\pm$ 4.29	82.42 $\pm$ 3.09	90.23 $\pm$ 1.70
Reason-Only	87.06 $\pm$ 1.76	92.29 $\pm$ 0.58	96.87 $\pm$ 0.26	93.06 $\pm$ 1.96	93.06 $\pm$ 1.96	85.76 $\pm$ 2.39	92.12 $\pm$ 0.95
Reason-Hierarchical	89.30 $\pm$ 1.53	92.54 $\pm$ 1.19	97.61 $\pm$ 0.33	90.28 $\pm$ 1.96	91.67 $\pm$ 0.00	89.09 $\pm$ 1.48	92.73 $\pm$ 1.45
Reason-Replan	92.54 $\pm$ 1.06	95.52 $\pm$ 0.61	97.34 $\pm$ 0.58	100.00 $\pm$ 0.00	100.00 $\pm$ 0.00	90.91 $\pm$ 1.29	94.55 $\pm$ 0.74
Ours	97.51$\pm$0.35	97.64$\pm$0.18	99.01$\pm$0.28	100.00$\pm$0.00	100.00$\pm$0.00	96.97$\pm$0.43	97.12$\pm$0.21

TABLE II

ENVIRONMENT INTERACTION AND TOKEN EFFICIENCY ON THE AGENTBOARD ALFWORLD OOD SPLIT WITH QWEN3-14B. RESULTS ARE REPORTED AS MEAN $\pm$ STANDARD DEVIATION OVER THREE RUNS.

Method	Env. steps $\downarrow$	Tokens / episode $\downarrow$
Plan-Only	20.61 $\pm$ 0.47	31,676 $\pm$ 875
Reason-Only	14.98 $\pm$ 0.23	54,713 $\pm$ 2,629
Reason-Hierarchical	17.25 $\pm$ 0.10	35,950 $\pm$ 25
Reason-Replan	17.29 $\pm$ 0.11	8,422$\pm$104
Ours	10.14$\pm$0.13	25,191 $\pm$ 171

progress steps satisfied during execution, and Grounding Accuracy (GA) measures the proportion of generated actions that are valid under the environment state at the time of execution.

- **Performance by Task Difficulty.** We report SR and PR separately on the Easy and Hard subsets.
- **Efficiency.** We report the per-episode averages of environment interactions and total tokens across all online LLM calls during execution.

B. Main Results

Tables I and II report task performance and efficiency, respectively, while Fig. 4 shows representative ProLoop executions in ALFWORLD. **1) Overall Task Performance.** Compared with the strongest corresponding baselines, ProLoop improves overall SR by 12.94% and 4.97%, and PR by 6.18% and 2.12% under Qwen3-14B and GPT-5.6, respectively. It also achieves the highest GA under both backbones, reaching 98.23% and 99.01%. These results show that explicit progress assessment and adaptation improve reliable long-horizon execution. Notably, ProLoop with Qwen3-14B outperforms all four GPT-5.6 baseline methods in overall performance, showing that effective progress-aware execution can provide substantial gains beyond those obtained from a stronger backbone alone. **2) Performance by Task Difficulty.** On the 24 Easy tasks, ProLoop with Qwen3-14B

achieves 95.83% SR and 98.61% PR, slightly below Reason-Replan at 100%, while GPT-5.6 ProLoop matches the best result at 100% on both metrics. The advantage becomes more pronounced on the 110 Hard tasks, where ProLoop improves SR by 15.46% and 6.06%, and PR by 6.82% and 2.57% under the two backbones, respectively. Since Hard tasks require more subgoals and transition decisions, the larger gains suggest that explicit progress regulation is particularly beneficial when coordinating more intermediate objectives. **3) Efficiency.** ProLoop requires only 10.14 environment interactions per episode, the lowest among all methods. Compared with Reason-Only, it reduces environment interactions by 32.31% and token usage by 53.96%. Its 25,191 tokens per episode are the second lowest overall, indicating a favorable balance between task performance and resource efficiency.

VI. ANALYSIS AND DISCUSSION

A. Subgoal Transition Reliability

We compare ProLoop with the *Implicit LLM* variant (w/o *Progress Loop* in Table IV). This variant retains the ReAct-style executor and cross-subgoal memory but infers completion from the accumulated trajectory and outputs **Completed** to trigger transition. Across three seeds, subgoals are retrospectively labeled by humans with LLM assistance, excluding those with neither completion nor transition before episode termination. Transitions are *on-time*, *delayed*, or *premature* if triggered immediately upon completion, at a later step, or before completion, respectively. A transition is *missed* if no transition occurs before episode termination despite subgoal completion.

ProLoop achieves 98.66% on-time transitions versus 68.70% for the Implicit LLM (Table III). Only 1.34% are delayed, with no premature or missed transitions observed in the evaluated runs. These results suggest that explicit progress assessment supports continued execution until completion and timely transitions thereafter, enabling reliable closed-loop adaptation.



Fig. 5. Real-world execution of a breakfast preparation task with ProLoop on the dual-arm mobile robot.

TABLE III

SUBGOAL TRANSITION RELIABILITY UNDER EXPLICIT AND IMPLICIT PROGRESS ASSESSMENT WITH QWEN3-14B. ALL VALUES ARE PERCENTAGES COMPUTED FROM POOLED COUNTS ACROSS THREE RUNS WITH DIFFERENT RANDOM SEEDS.

Method	On-time ↑	Delayed ↓	Premature ↓	Missed ↓
Implicit LLM	68.70	11.94	16.08	3.29
Ours	98.66	1.34	0.00	0.00

B. Ablation Study

We evaluate component contributions and complementary effects through ablations (Table IV). In **w/o Progress Loop**, Structured State Construction, State-Relation Monitoring, and Progress-Guided Adaptation are removed. Overall and Hard-task SR decrease by 11.45% and 13.94%, respectively, while Easy-task SR remains unchanged, suggesting that explicit progress regulation is more important when coordinating more subgoals and transitions. In **w/o Summary**, completion-gated summaries are replaced with the full local trajectory τ_k . Overall and Hard-task SR drop by 3.98% and 5.76%, despite a slight Easy-task gain, suggesting that selective summarization becomes more valuable as cross-subgoal context accumulates. The **w/o Both** variant combines these changes, further reducing overall and Hard-task SR to 83.58% and 80.91%, confirming that progress regulation and completion-gated memory provide complementary benefits. Finally, **w/o Relation Head** replaces the learned relation head with direct LLM judgment under the same desired conditions and structured execution states. Overall and Hard-task SR decrease by 9.95% and 12.12%, respectively, supporting the benefit of learned relation assessment beyond structured state representation alone.

C. Real-World Robotic Experiment

We further validate ProLoop on a dual-arm mobile robot equipped with a RealSense D435i head camera, using Qwen3.8-flash as the VLM backbone. The robot executes

TABLE IV

ABLATION STUDY OF PROLOOP COMPONENTS ON THE AGENTBOARD ALFWORLD OOD SPLIT WITH QWEN3-14B. RESULTS ARE AVERAGED OVER THREE RUNS.

Method	Overall (%)		Easy (%)	Hard (%)
	SR ↑	PR ↑	SR ↑	SR ↑
Ours full	96.52	97.39	95.83	96.67
w/o Progress Loop	85.07	91.92	95.83	82.73
w/o Summary	92.54	96.08	100.00	90.91
w/o Both	83.58	93.97	95.83	80.91
w/o Relation Head	86.57	93.97	95.83	84.55

semantic subgoals through four atomic skills: **Grasp**, **Place**, **Move_to**, and **Observe_state**. We evaluate ProLoop on two representative tasks, breakfast preparation and table-top tidying, with 10 trials for each task. Across 20 trials, ProLoop achieves an overall success rate of 90%. Fig. 5 illustrates a representative six-subgoal breakfast preparation task, showing how progress assessment supports continued execution, correction, and subgoal transition throughout the task. Additional details and demonstrations of the real-world experiments are available on our project website and in the supplementary videos.

VII. CONCLUSION

We present **ProLoop**, a progress-aware closed-loop framework for long-horizon robotic task planning. ProLoop derives subgoal progress from structured relations between desired conditions and evolving execution states. This progress guides timely closed-loop adaptation by preserving ongoing execution, triggering correction upon deviation, and permitting transition upon completion. Completion-gated memory retains task-relevant information for subsequent planning. Experiments on ALFWORLD show improved task success with fewer environment interactions, while real-world trials demonstrate feasibility on a physical robot. These results highlight the value of explicit progress assessment for reliable long-horizon execution.

REFERENCES

- [1] I. Singh, V. Blukis, A. Mousavian, A. Goyal, D. Xu, J. Tremblay, D. Fox, J. Thomason, and A. Garg, “ProgPrompt: Generating situated robot task plans using large language models,” in *2023 IEEE International Conference on Robotics and Automation*, 2023, pp. 11 523–11 530.
- [2] J. Zhang, L. Tang, Y. Song, Q. Meng, H. Qian, J. Shao, W. Song, S. Zhu, and J. Gu, “FLTRNN: Faithful long-horizon task planning for robotics with large language models,” in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 6680–6686.
- [3] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, “Understanding the planning of LLM agents: A survey,” *arXiv preprint arXiv:2402.02716*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.02716>
- [4] T. Chakraborty, U. Ghosh, X. Zhang, F. F. Niloy, Y. Dong, J. Li, A. Roy-Chowdhury, and C. Song, “HEAL: An empirical study on hallucinations in embodied agents driven by large language models,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 21 226–21 243. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.1158/>
- [5] A. Ajay, S. Han, Y. Du, S. Li, A. Gupta, T. S. Jaakkola, J. B. Tenenbaum, L. P. Kaelbling, A. Srivastava, and P. Agrawal, “Compositional foundation models for hierarchical planning,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 22 304–22 325. [Online]. Available: <https://doi.org/10.52202/075280-0979>
- [6] Y. Liu, L. Palmieri, S. Koch, I. Georgievski, and M. Aiello, “DELTA: Decomposed efficient long-term robot task planning using large language models,” in *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 2025, pp. 10 995–11 001.
- [7] M. Hu, T. Chen, Q. Chen, Y. Mu, W. Shao, and P. Luo, “HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Vienna, Austria: Association for Computational Linguistics, 2025, pp. 32 779–32 798. [Online]. Available: <https://aclanthology.org/2025.acl-long.1575/>
- [8] M. Kim and S.-w. Hwang, “CoEx – co-evolving world-model and exploration,” in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 21 629–21 651. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.1179/>
- [9] H. Sun, Y. Zhuang, L. Kong, B. Dai, and C. Zhang, “AdaPlanner: Adaptive planning from feedback with language models,” in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 58 202–58 245. [Online]. Available: <https://doi.org/10.52202/075280-2537>
- [10] J. Kim, S. Rhee, M. Kim, D. Kim, S. Lee, Y. Sung, and K. Jung, “ReflAct: World-grounded decision making in LLM agents via goal-state reflection,” in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. Suzhou, China: Association for Computational Linguistics, 2025, pp. 33 433–33 465. [Online]. Available: <https://aclanthology.org/2025.emnlp-main.1697/>
- [11] Y. Farag, S. Stoyanchev, M. Li, S. Keizer, and R. Doddipatla, “Conditional multi-stage failure recovery for embodied agents,” *arXiv preprint arXiv:2507.06016*, 2025, accepted at REALM 2025. [Online]. Available: <https://arxiv.org/abs/2507.06016>
- [12] C. Ma, J. Zhang, Z. Zhu, C. Yang, Y. Yang, Y. Jin, Z. Lan, L. Kong, and J. He, “AgentBoard: An analytical evaluation board of multi-turn LLM agents,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 74 325–74 362.
- [13] S. Li, X. Puig, C. Paxton, Y. Du, C. Wang, L. Fan, T. Chen, D.-A. Huang, E. Akyürek, A. Anandkumar *et al.*, “Pre-trained language models for interactive decision-making,” in *Advances in Neural Information Processing Systems*, vol. 35, 2022, pp. 31 199–31 212.
- [14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. R. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *The Eleventh International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X
- [15] J. Liang, W. Huang, F. Xia, P. Xu, K. Hausman, B. Ichter, P. Florence, and A. Zeng, “Code as policies: Language model programs for embodied control,” in *2023 IEEE International Conference on Robotics and Automation*, 2023, pp. 9493–9500. [Online]. Available: <https://ieeexplore.ieee.org/document/10160591/>
- [16] B. Ichter, A. Brohan, Y. Chebotar, C. Finn, K. Hausman, A. Herzog, D. Ho, J. Ibarz, A. Irpan, E. Jang *et al.*, “Do as I can, not as I say: Grounding language in robotic affordances,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–318. [Online]. Available: <https://proceedings.mlr.press/v205/ichter23a.html>
- [17] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, “LLM-Planner: Few-shot grounded planning for embodied agents with large language models,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, Oct. 2023, pp. 2998–3009.
- [18] T. Zhou, Z. Wang, H. Ao, G. Chen, B. Xing, J. Cheng, Y. Yang, and Y. Yue, “STEP planner: Constructing cross-hierarchical subgoal tree as an embodied long-horizon task planner,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2025, pp. 16 731–16 738.
- [19] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar *et al.*, “Inner monologue: Embodied reasoning through planning with language models,” in *Proceedings of the 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 1769–1782. [Online]. Available: <https://proceedings.mlr.press/v205/huang23c.html>
- [20] S. Nayak, A. Morrison Orozco, M. Ten Have, V. Thirumalai, J. Zhang, D. Chen, A. Kapoor, E. Robinson, K. Gopalakrishnan, J. Harrison *et al.*, “Long-horizon planning for multi-agent robots in partially observable environments,” in *Advances in Neural Information Processing Systems*, vol. 37, 2024, pp. 67 929–67 967. [Online]. Available: <https://doi.org/10.52202/079017-2169>
- [21] Z. Zheng, Q. Feng, H. Li, A. Knoll, and J. Feng, “Evaluating uncertainty-based failure detection for closed-loop LLM planners,” *arXiv preprint arXiv:2406.00430*, 2024, accepted at the ICRA 2024 Workshop on Back to the Future: Robot Learning Going Probabilistic. [Online]. Available: <https://arxiv.org/abs/2406.00430>
- [22] M. Shridhar, X. Yuan, M.-A. Côté, Y. Bisk, A. Trischler, and M. Hausknecht, “ALFWorld: Aligning text and embodied environments for interactive learning,” in *International Conference on Learning Representations*, 2021. [Online]. Available: <https://alfworld.github.io/>